Trabajo Práctico N° 4

Codificación de fuentes de información

Primer Teorema de Shannon. Rendimiento y redundancia de un código. Códigos de Huffman. Código de Shannon-Fano. Codificación para control de errores. Códigos bloque. Detección y corrección de errores por chequeo de paridad. Estrategias avanzadas de control: Códigos BCH. Códigos Reed-Solomon. Distancia de Levenshtein.

1. Codificar una función booleana en Python que reciba como parámetros: una lista con la distribución de probabilidades de una fuente, otra lista con palabras código para la extensión de orden N y el valor de N, y verifique si el código cumple con el Primer Teorema de Shannon.
2. Dada la siguiente fuente de información y su codificación, comprobar si la fuente y su extensión de orden 2 cumplen con el Primer Teorema de Shannon:

Fuente	Probs	Código
S1	0.3	BA
S2	0.1	CAB
S3	0.4	A
S4	0.2	CBA

3. Si se tiene una fuente de información con probabilidades P = { 0.5, 0.2, 0.3 } y los siguientes códigos para la fuente y su extensión de orden 2, verificar el cumplimiento del Primer Teorema de Shannon para ambas codificaciones:
   - C1 = { 11, 010, 00 }
   - C2 = { 10, 001, 110, 010, 0000, 0001, 111, 0110, 0111 }
4. Para una fuente binaria con ω = 0.8, calcular la extensión de orden 3 y proponer una codificación binaria que cumpla con el Primer Teorema de Shannon.
5. Para una fuente binaria con ω = 0.7:
   a. Obtener una codificación mediante el algoritmo de Huffman
   b. Codificar la extensión de orden 2 mediante el algoritmo de Shannon-Fano
   c. Comprobar si las codificaciones cumplen con el Primer Teorema de Shannon
6. Realizar una función en Python que reciba como parámetros: dos listas paralelas con la distribución de probabilidades de una fuente y su codificación, y calcule el rendimiento y la redundancia del código.
7. Comparar los rendimientos y las redundancias de los códigos del ejercicio 3.

8. Comparar los rendimientos y las redundancias de los siguientes códigos:

Fuente	Probs	Código 1	Código 2	Código 3	Código 4
A	0.2	01	00	0110	11
B	0.15	111	01	010	001
C	0.1	110	10	0111	000
D	0.3	101	110	1	10
E	0.25	100	111	00	01

9. Dados los siguientes códigos que representan colores:

Color	Código 1	Código 2	Código 3
Rojo	00	000	0000
Amarillo	01	100	0011
Verde	10	101	1010
Azul	11	111	0101

   a. Calcular la distancia de Hamming de cada código
   b. Determinar para cada código cúantos errores se pueden detectar y corregir

10. Implementar una función en Python que reciba una lista de cadenas de caracteres que representa una codificación binaria y devuelva: la distancia de Hamming, la cantidad de errores que se pueden detectar y la cantidad de errores que se pueden corregir.
11. Dados los siguientes códigos que representan colores:

Color	Código 1	Código 2	Código 3
Rojo	0100100	0100100	0110000
Amarillo	0101000	0010010	0000011
Verde	0010010	0101000	0101101
Azul	0100000	0100001	0100110

   a. Obtener sus distancias de Hamming
   b. Especificar cúantos errores se pueden detectar y corregir en cada caso

12. Desarrollar funciones en Python que resuelvan lo siguiente:
   a. Dado un carácter, devolver un byte que represente su código ASCII (7 bits) y utilice el bit menos significativo para almacenar la paridad del código.
   b. Dado un byte que se obtuvo como resultado de la función anterior, verificar si es correcto o tiene errores.

13. Dadas las siguientes matrices que contienen mensajes representados con código ASCII y sus bits de paridad vertical, longitudinal y cruzada, detectar los errores y, en caso de ser posible, recuperar el mensaje original:

a.
```
00100001
10000111
10000010
10100110
10000010
```

b.
```
00101101
10011001
10001010
10011100
10000010
```

c.
```
00101010
10000010
10011010
10011111
10100101
```

d.
```
00010100
10010000
10011110
10011001
10000010
```

e.
```
00110101
10011010
10101011
10100100
10000010
```

f.
```
00001001
10101001
10100101
10001011
10100110
```

g.
```
00011101
10010011
10011101
10001100
10011111
```

h.
```
00111110
10000111
10010000
10000010
10101010
```

14. Codificar funciones en Python que resuelvan lo siguiente:
    a. Dada una cadena de caracteres, generar una secuencia de bytes (bytearray) que contenga su representación con código ASCII y sus bits de paridad vertical, longitudinal y cruzada.
    b. Dada una secuencia de bytes que se obtuvo como resultado de la función anterior, devolver el mensaje original o una cadena de caracteres vacía si no se pueden corregir los errores.

<u>Sugerencia:</u> generar una matriz de bits para realizar las operaciones, transformando la secuencia de bytes en una lista de cadenas de caracteres binarias y, luego, cada cadena de caracteres en una lista de números enteros que representen los bits.

Resultados:

2.	n = 1	$H_3(S) = 1.16$	$L_1 = 1.9$	Cumple
	n = 2	$H_3(S) = 1.16$	$L_2 = 3.8$	No cumple
3.	n = 1	$H_2(S) = 1.49$	$L_1 = 2.2$	Cumple
	n = 2	$H_2(S) = 1.49$	$L_2 = 3$	Cumple

5. a. $L_1 = \{ 1, 1 \}$
   b. $L_2 = \{ 1, 3, 2, 3 \}$

c.	n = 1	$H_2(S) = 0.88$	$L_1 = 1.00$	Cumple
	n = 2	$H_2(S) = 0.88$	$L_2 = 1.81$	Cumple

7. $\eta_1 = 0.6752$ $\eta_2 = 0.9903$
   $R_1 = 0.3248$ $R_2 = 0.0097$

8. $\eta_1 = 0.7958$ $\eta_2 = 0.8738$ $\eta_3 = 0.9095$ $\eta_4 = 0.9903$
   $R_1 = 0.2042$ $R_2 = 0.1262$ $R_3 = 0.0905$ $R_4 = 0.0097$

9. a. $d_1 = 1$ $d_2 = 1$ $d_3 = 2$

   b. **Código 1:** no se pueden detectar ni corregir errores

   Código 2: no se pueden detectar ni corregir errores

   Código 3: se puede detectar un solo error pero no corregir ninguno

11. a. $d_1 = 1$ $d_2 = 2$ $d_3 = 3$

   b. **Código 1:** no se pueden detectar ni corregir errores

   Código 2: se puede detectar un solo error pero no corregir ninguno

   Código 3: se pueden detectar dos errores pero solo corregir uno

13. a. CASA b. LUNA c. — d. —

   e. — f. — g. — h. —